

COMPETENCY-BASED LEARNING MATERIALS (CBLM)

AI BASIC PYTHON DEVELOPER

TECHNICAL AND VOCATIONAL CERTIFICATE I — LEVEL II

Core Competency: Apply Structured Computer Programming

Module Title: Apply Structured Computer Programming

Learning Outcome: LO8 – Develop a Program using Sets

Nominal Duration:

LEARNING OUTCOME 08

Develop a Program using Sets

Upon completion of this Learning Outcome, students/trainees will have the following competencies:

1. Create and initialize Python sets according to program requirements.
2. Access and display set elements using iteration techniques.
3. Apply set operations including union, intersection, difference and symmetric difference.
4. Use appropriate set methods, keywords and constructors.
5. Add and remove set items using appropriate methods.
6. Use built-in functions with sets to process data.
7. Develop Python programs using sets for storing unique data.
8. Test and debug set operations to ensure correct program execution.

INFORMATION SHEET No. 8.1: Introduction to Python Sets

Objective

Upon reading this Information Sheet, the trainee must be able to:

- Explain the properties and characteristics of Python sets.
- Describe the syntax and structure of sets.
- Declare and create Python sets.
- Access and iterate set elements.
- Distinguish sets from other Python data structures.
- Use set methods, keywords and constructors.
- Explain unique values and duplicate item removal.
- Add and remove elements from sets.
- Perform union, intersection, difference and symmetric difference.
- Apply built-in functions with sets.
- Explain practical applications of sets for data processing.

1. What is a Python Set?

A Python set is a collection used to store unique values. A set is useful when a program needs to keep one copy of each value and when mathematical set operations are required.

Important characteristics of Python sets include: unordered storage, no numeric indexing, unique elements, and support for adding and removing elements.

Example:

```
skills = {"Python", "Database", "Networking"}  
print(skills)
```

2. Properties and Characteristics of Sets

A set stores unique elements. If the same value is entered more than once, only one copy is retained. Sets are also unindexed, so elements are not accessed with positions such as [0] or [-1].

The displayed order of elements may vary. Therefore, trainees should judge set results by the values present rather than by a fixed display order.

Example:

```
values = {10, 20, 20, 30, 30}  
print(values)  
  
# Result contains 10, 20 and 30 only.
```

3. Set Syntax and Structure

A non-empty set is normally written using curly braces with comma-separated values. An empty set is created with set(), because {} creates an empty dictionary.

Example:

```
fruits = {"apple", "banana", "cherry"}  
  
empty_set = set()
```

4. Declaration and Creation of Sets

Sets can be created directly with literals or by using the `set()` constructor to convert another iterable such as a list or tuple.

Example:

```
numbers = {1, 2, 3, 4}  
  
from_list = set([1, 2, 2, 3, 4])  
  
from_tuple = set((10, 20, 20, 30))
```

5. Accessing and Iterating Set Elements

Because a set has no numeric indexes, elements are normally accessed with a `for` loop. Membership operators can be used to check whether a value exists.

Example:

```
subjects = {"Python", "Database", "AI"}  
  
for subject in subjects:  
    print(subject)  
  
print("Python" in subjects)
```

6. Sets Compared with Other Python Data Structures

Lists and tuples are sequences and support indexing. Sets do not support numeric indexing and automatically remove duplicate values. Dictionaries store key-value pairs and have unique keys.

Feature	List	Tuple	Set	Dictionary
Ordered	Yes	Yes	No guaranteed order	Yes
Indexed	Yes	Yes	No	By key
Duplicates	Allowed	Allowed	Not allowed	Keys unique
Changeable	Yes	No	Yes	Yes
Common syntax	<code>[]</code>	<code>()</code>	<code>{}</code>	<code>{key: value}</code>

7. Set Methods, Keywords and Constructors

Python provides methods for adding, removing, comparing and copying set data. Important keywords and operators include `in`, `not in`, `|`, `&`, `-`, and `^`.

8. Unique Values and Duplicate Item Removal

A common practical use of sets is removing duplicate values from a list or other collection.

Example:

```
names = ["Dara", "Sok", "Dara", "Rina", "Sok"]
unique_names = set(names)
print(unique_names)
```

9. Adding and Removing Set Elements

Use `add()` to add one item and `update()` to add multiple items. Use `remove()` when the item must exist, `discard()` when the program should continue safely if an item is absent, `pop()` to remove an arbitrary item, and `clear()` to remove all items.

Example:

```
fruits = {"apple", "banana"}
fruits.add("mango")
fruits.update(["orange", "melon"])
fruits.remove("banana")
fruits.discard("grape")
print(fruits)
```

10. Set Operations

Set operations allow a program to compare groups of values. Union returns all unique values from both sets. Intersection returns values common to both sets. Difference returns values found in the first set but not the second. Symmetric difference returns values that are in only one of the two sets.

Example:

```
set_a = {1, 2, 3, 4}
set_b = {3, 4, 5, 6}

print(set_a.union(set_b))
print(set_a.intersection(set_b))
print(set_a.difference(set_b))
print(set_a.symmetric_difference(set_b))
```

11. Built-in Functions Used with Sets

Built-in functions such as `len()`, `min()`, `max()` and `sum()` can be used with sets when the data type is suitable.

Example:

```
scores = {70, 80, 90}
print(len(scores))
print(min(scores))
```

```
print(max(scores))  
  
print(sum(scores))
```

12. Practical Applications of Sets for Data Processing

Sets are useful for removing duplicates, comparing two groups, finding common records, identifying unique records, checking membership and processing data where uniqueness is important.

Common set methods:

Method / Function	Purpose
set()	Creates a set or converts an iterable to a set
add()	Adds one item
update()	Adds multiple items
remove()	Removes an item and raises an error if it is absent
discard()	Removes an item without an error if it is absent
pop()	Removes an arbitrary item
clear()	Removes all items
union()	Combines unique items
intersection()	Returns common items
difference()	Returns items unique to the first set
symmetric_difference()	Returns non-common items
len()	Returns the number of items

13. Integrated Set Processing Example

The following example combines declaration, iteration, adding/removing items, built-in functions and set operations.

```
python_students = {"Dara", "Sok", "Rina", "Vanna"}  
database_students = {"Sok", "Rina", "Malis"}  
  
for student in python_students:  
    print(student)  
  
python_students.add("Sophy")  
python_students.discard("Vanna")  
  
print("Count:", len(python_students))  
print("Union:", python_students.union(database_students))  
print("Intersection:", python_students.intersection(database_students))
```

```
print("Difference:", python_students.difference(database_students))
```

14. Python Set Program Workflow

A typical set program follows a sequence from identifying the data requirement through creating, processing and verifying the set.

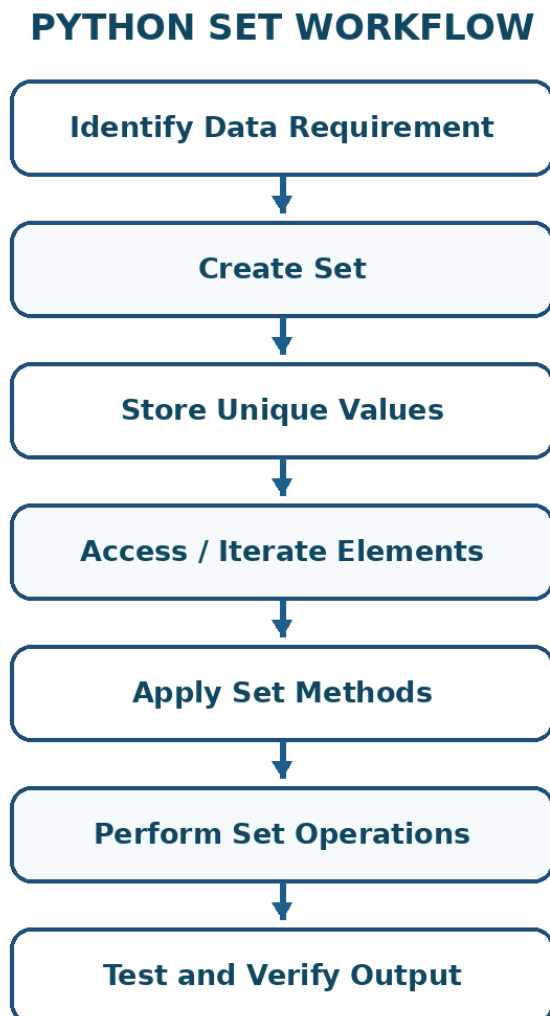


Figure 1: Python Set Workflow

The trainee should test the program using different values and verify the content of each result. Set output order is not used as the main correctness criterion.

SELF-CHECK No. 8.1

Instructions

Answer all questions. Do not refer to the Information Sheet while answering.

Part A: Multiple Choice

1. What is a main characteristic of a Python set?
A. It stores duplicate values B. It stores unique values
C. It supports positive indexing D. It is always sorted
2. Which statement creates an empty set?
A. {} B. [] C. set() D. ()
3. Which method adds one item to a set?
A. append() B. add() C. insert() D. push()
4. Which method can add multiple items to a set?
A. update() B. add() C. append() D. extenditem()
5. Which method removes an item without raising an error if it is absent?
A. remove() B. discard() C. delete() D. erase()
6. Which operation returns values common to two sets?
A. Union B. Difference C. Intersection D. Symmetric difference
7. Which operation combines all unique values from two sets?
A. Union B. Difference C. Intersection D. clear()
8. Which function returns the number of set elements?
A. size() B. count() C. len() D. length()
9. How are set elements normally accessed?
A. Using numeric indexes B. Using iteration
C. Using negative indexes D. Using slicing
10. Which constructor can convert a list to a set?
A. list() B. tuple() C. set() D. dict()

ANSWER KEY No. 8.1

Part A

No.	Answer
1	B
2	C
3	B
4	A
5	B
6	C
7	A
8	C
9	B
10	C

OPERATION SHEET No. 8.1:

Title: Create, Process and Verify Python Sets

Objective

Upon completion of this Operation Sheet, the trainee must be able to:

- Create and initialize Python sets.
- Access set elements using iteration.
- Add and remove set items.
- Use set methods and constructors.
- Perform set operations.
- Apply built-in functions.
- Execute and verify a set program.

Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Desktop Computer/Laptop	Unit	1
2	Windows Operating System	Unit	1
3	Python 3	Copy	1
4	Python IDE/Code Editor	Software	1
5	CBLM	Copy	1
6	Worksheet	Copy	1

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Open the Python IDE	Development environment opens correctly
2	Create a new Python file	.py file is created
3	Declare a set	Correct set syntax is used
4	Create an empty set	set() is used correctly
5	Store values	Unique values are retained
6	Iterate through the set	Elements are displayed using a for loop
7	Check membership	in / not in produces correct result
8	Add one item	add() works correctly
9	Add multiple items	update() works correctly
10	Remove an item	Appropriate removal method is used
11	Create a second set	Second set is declared correctly
12	Perform union	All unique values are returned
13	Perform intersection	Common values are returned
14	Perform difference	Correct first-set-only values are returned
15	Perform symmetric difference	Non-common values are returned
16	Apply len()	Correct number of elements is displayed

No.	Procedure/Step	Key Performance
17	Run the program	Program executes without syntax errors
18	Verify output	Result matches program requirements

Detailed Operation Procedure

Step 1: Open the development environment

1. Start the computer.
2. Open IDLE, Visual Studio Code, PyCharm or another approved Python environment.
3. Create a new Python file and save it in the assigned folder.

Step 2: Declare sets

1. Create a non-empty set using curly braces.
2. Create an empty set using set().
3. Enter sample data including one duplicate value and observe the result.

Step 3: Access set elements

1. Use a for loop to display each element.
2. Use the in operator to test whether a specific value exists.

Step 4: Add items

1. Use add() for one value.
2. Use update() for several values.

Step 5: Remove items

1. Use remove() for an item known to exist.
2. Use discard() when an item may not exist.
3. Use pop() only when arbitrary removal is acceptable.

Step 6: Apply set operations

1. Create a second set.
2. Apply union, intersection, difference and symmetric difference.

Step 7: Apply built-in functions

1. Use len() to count elements.
2. Use min(), max() or sum() when appropriate for numeric sets.

Step 8: Test and verify

1. Run the program.
2. Test with different values.
3. Correct syntax or logic errors.
4. Verify that each set result contains the expected elements.

Safety Precautions

1. Use an authorized computer.
2. Use approved Python software and files.
3. Do not modify unrelated system settings.
4. Protect user and institutional information.
5. Save files in the assigned location.
6. Read error messages carefully before changing code.
7. Report equipment or software problems to the trainer.
8. Follow laboratory rules.

TASK SHEET No. 8.1

Title: Declare, Access and Iterate Python Sets

Performance Task:

The trainee shall create a Python set containing student skills, access the elements through iteration and verify membership.

Performance Objective:

Given a computer with Python installed, a Python IDE and the required CBLM, the trainee must create, access and verify a Python set correctly.

Description:

The trainee will perform individual practice activities to declare a set, observe unique values, iterate through the set and check whether a required value exists.

Picture/Overview:

SET ACCESS & ITERATION WORKFLOW

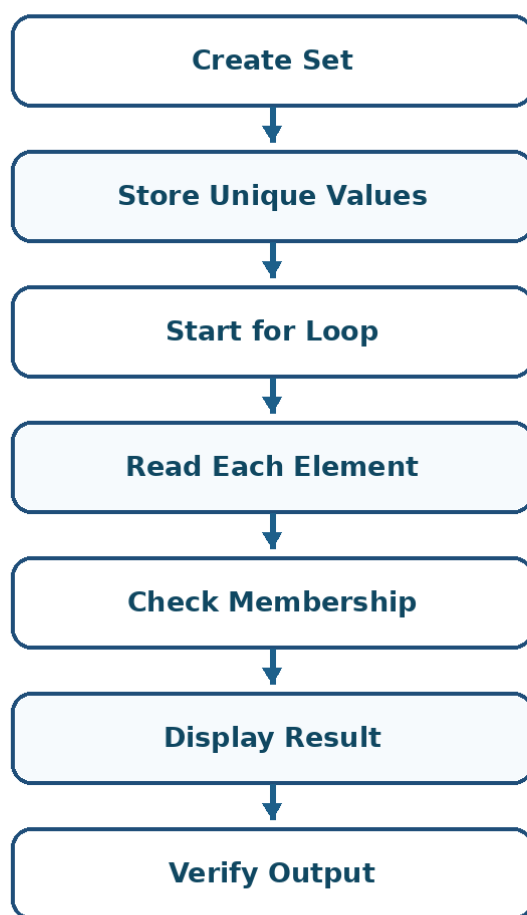


Figure 2: Set Access and Iteration Workflow

Tools, Equipment and Materials:

No.	Description	Unit	Qty
1	Desktop Computer/Laptop	Unit	1
2	Python 3	Copy	1
3	Python IDE/Code Editor	Software	1
4	CBLM	Copy	1

No.	Description	Unit	Qty
5	Worksheet	Copy	1

Supplier:

- Training institution
- Authorized software provider
- Official Python software source

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Open Python IDE	IDE opens successfully
2	Create a new file	.py file is created
3	Declare a set	Correct set syntax is used
4	Include one duplicate value	Only one unique copy is retained
5	Use a for loop	All elements are iterated
6	Check membership	Correct True/False result is produced
7	Display set length	Correct len() result is produced
8	Verify output	Results contain expected values

Practical Activities

Activity 1: Create and display a set

```
skills = {"Python", "Database", "Networking", "Python"}
print(skills)
```

Activity 2: Iterate the set

```
for skill in skills:
    print(skill)
```

Activity 3: Check membership and length

```
print("Python" in skills)
print("AI" in skills)
print(len(skills))
```

Assessment Method:

- Direct observation
- Practical demonstration
- Oral questioning
- Performance checklist

TASK SHEET No. 8.2

Title: Add and Remove Python Set Items

Performance Task:

The trainee shall modify a Python set using appropriate add and remove methods.

Performance Objective:

Given a computer with Python installed and a Python IDE, the trainee must add and remove set items correctly and verify the updated set.

Description:

The trainee will practice `add()`, `update()`, `remove()`, `discard()`, `pop()` and `clear()`, and will explain when each method is appropriate.

Picture/Overview:

SET ITEM MANAGEMENT WORKFLOW

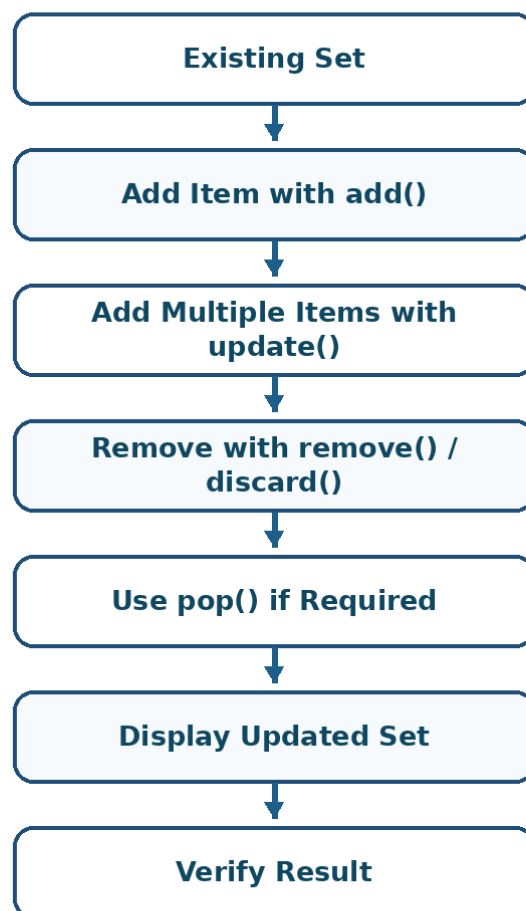


Figure 3: Set Item Management Workflow

Tools, Equipment and Materials:

No.	Description	Unit	Qty
1	Desktop Computer/Laptop	Unit	1
2	Python 3	Copy	1
3	Python IDE/Code Editor	Software	1

No.	Description	Unit	Qty
4	CBLM	Copy	1
5	Worksheet	Copy	1

Supplier:

- Training institution
- Authorized software provider
- Official Python software source

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Create a set	Starting values are correct
2	Add one item with add()	One new item is added
3	Add several items with update()	All required items are added
4	Remove an existing item with remove()	Required item is removed
5	Use discard() for an optional item	Program continues safely if item is absent
6	Use pop()	Trainee understands arbitrary removal
7	Display updated set	Current values are shown
8	Verify result	Updated set meets requirements

Practical Activity

```
fruits = {"apple", "banana", "cherry"}
```

```
fruits.add("mango")
fruits.update(["orange", "melon"])
fruits.remove("banana")
fruits.discard("grape")
```

```
print(fruits)
```

Then repeat the exercise with different item names and record the final set.

Assessment Method:

- Direct observation
- Practical demonstration
- Review of .py file
- Output verification
- Performance checklist

TASK SHEET No. 8.3

Title: Apply Python Set Operations

Performance Task:

The trainee shall compare two sets using union, intersection, difference and symmetric difference.

Performance Objective:

Given two sets of student data, the trainee must apply the required set operations and interpret each result correctly.

Description:

The trainee will compare two course-registration sets and identify all students, common students, course-specific students and students found in only one course.

Picture/Overview:

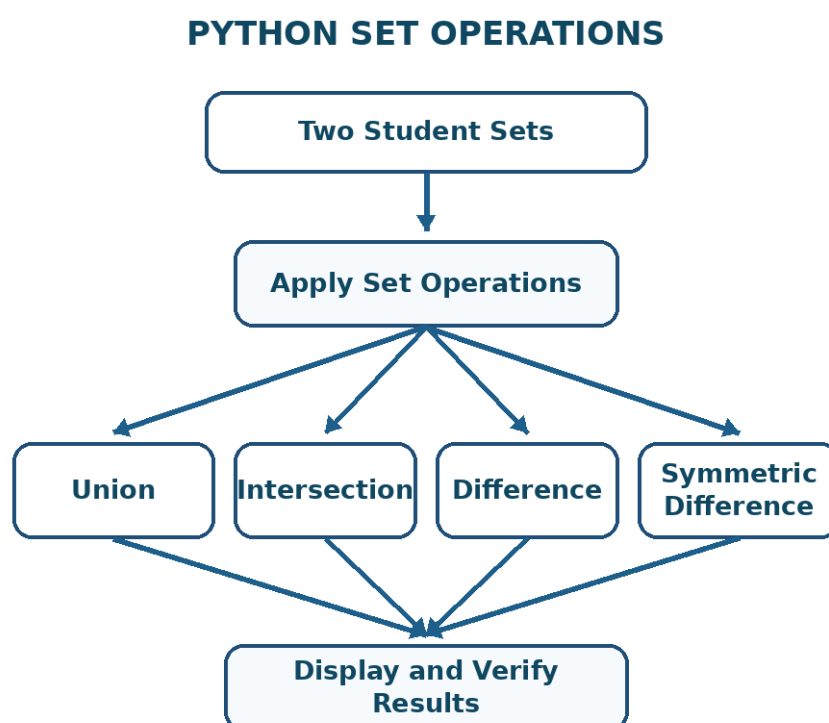


Figure 4: Python Set Operations

Tools, Equipment and Materials:

No.	Description	Unit	Qty
1	Desktop Computer/Laptop	Unit	1
2	Python 3	Copy	1
3	Python IDE/Code Editor	Software	1
4	CBLM	Copy	1

No.	Description	Unit	Qty
5	Worksheet	Copy	1

Supplier:

- Training institution
- Authorized software provider
- Official Python software source

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Create Python-student set	Correct student values are stored
2	Create Database-student set	Correct student values are stored
3	Apply union	All unique students are returned
4	Apply intersection	Students in both courses are returned
5	Apply difference	Python-only students are returned
6	Apply symmetric difference	Students in only one course are returned
7	Display results	Labels and results are readable
8	Verify output	Each operation is interpreted correctly

Practical Activity

```
python_students = {"Dara", "Sok", "Rina", "Vanna"}
database_students = {"Sok", "Rina", "Malis"}

print("Union:", python_students.union(database_students))
print("Intersection:", python_students.intersection(database_students))
print("Difference:", python_students.difference(database_students))
print("Symmetric Difference:", python_students.symmetric_difference(database_students))
```

Assessment Method:

- Observation
- Oral questioning
- Practical demonstration
- Output verification
- Performance checklist

WORKSHEET No. 8.1

Title: Python Sets Practice

Part A: Identify the Concept

Statement	Trainee Answer
A collection that stores unique values	_____
Function used to create an empty set	_____
Method used to add one item	_____
Method used to remove safely if an item may be missing	_____
Operation that returns common values	_____
Built-in function used to count set elements	_____

Part B: Complete the Code

```
fruits = {"apple", "banana", "orange"}
```

```
fruits._____("mango")
```

```
fruits._____("banana")
```

```
for fruit in fruits:
```

```
    print(_____)
```

Part C: Record the Set Operation Results

```
A = {1, 2, 3, 4}
```

```
B = {3, 4, 5, 6}
```

Operation	Result
A.union(B)	_____
A.intersection(B)	_____
A.difference(B)	_____
A.symmetric_difference(B)	_____

Part D: Practical Exercise

Create a set containing Python, Database, Networking, Web Development and AI. Then complete the following activities:

1. Add Cybersecurity.
2. Remove Networking.
3. Display all elements using iteration.
4. Determine the number of elements.
5. Check whether Python exists.

6. Save the file as skills_set.py.
7. Execute the program and record the result.

Recorded Result: _____

Trainee reflection: What worked, what failed, and what correction was applied?

JOB SHEET No. 8.1

Title: Develop a Student Course Set Management Program

Performance Task:

The trainee shall independently develop, test and verify a Python program that uses sets to process unique student registration data.

Performance Objective:

Given a computer, Python 3, a Python IDE, CBLM and training requirements, the trainee must develop a working set-processing program that demonstrates declaration, iteration, item modification, set operations, built-in functions, testing and debugging.

Description:

You have been asked to prepare a small program for comparing students registered in Python and Database courses. The program must:

1. Create two sets of student names.
2. Display the Python-student set through iteration.
3. Add one new student.
4. Remove one student using an appropriate method.
5. Check whether a specified student exists.
6. Determine the number of Python students.
7. Perform union.
8. Perform intersection.
9. Perform difference.
10. Perform symmetric difference.
11. Display clear results.
12. Run, test and debug the program.
13. Verify the final output.



Figure 5: Job Sheet Set Program Flow

Tools, Equipment and Materials:

No.	Description	Unit	Qty
1	Desktop Computer/Laptop	Unit	1
2	Windows Operating System	Unit	1
3	Python 3	Copy	1
4	Python IDE/Code Editor	Software	1
5	CBLM	Copy	1
6	Worksheet	Copy	1
7	USB Flash Drive	Unit	1

Supplier:

- Training institution
- Authorized software provider
- Official Python software source

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Analyze program requirements	Required set processing is correctly identified
2	Open the Python IDE	Development environment is ready
3	Create a new .py file	Program file is saved correctly
4	Create Python-student set	Correct set syntax is used
5	Create Database-student set	Second set is created correctly
6	Store unique names	Duplicate names are not retained
7	Iterate Python-student set	Each element can be displayed
8	Add a student	add() is used correctly
9	Remove a student	Appropriate removal method is used
10	Check membership	in / not in produces correct result
11	Determine length	len() produces correct result
12	Perform union	All unique students are returned
13	Perform intersection	Common students are returned
14	Perform difference	Python-only students are returned
15	Perform symmetric difference	Students in only one course are returned
16	Display labeled results	Output is clear and readable
17	Run program	Program executes successfully
18	Test with different data	Program continues to produce correct results
19	Debug errors	Syntax or logic errors are corrected
20	Verify final output	Output meets job requirements
21	Organize files	Source file and evidence are saved correctly
22	Apply safety and quality practices	Responsible laboratory practices are followed

Job Program

Create a file named:

```
student_course_sets.py
```

Enter:

```
python_students = {"Dara", "Sok", "Rina", "Vanna"}
```

```
database_students = {"Sok", "Rina", "Malis"}
```

```
print("Python Students")
```

for student in python_students:

```
    print(student)
```

```
python_students.add("Sophy")
```

```
python_students.discard("Vanna")
```

```
print("Count:", len(python_students))
```

```
print("Is Dara enrolled?", "Dara" in python_students)
```

```
print("All Students:", python_students.union(database_students))
```

```
print("Students in Both:", python_students.intersection(database_students))
```

```
print("Python Only:", python_students.difference(database_students))
```

```
print("Only One Course:", python_students.symmetric_difference(database_students))
```

Expected Output:

The exact display order of set items may vary. The required values for each operation must be correct, and the program must execute without unhandled errors.

Assessment Method:

- Direct observation
- Practical demonstration
- Product/output inspection
- Oral questioning
- Performance checklist

PERFORMANCE CHECKLIST No. 8.1

Task: Develop a Student Course Set Management Program

Name: _____

Date: _____

Did you use all these performance criteria during your task?

No.	Criteria	Yes	No
1	Declared Python sets correctly	☐	☐
2	Created an empty set with set() when required	☐	☐
3	Stored unique values correctly	☐	☐
4	Accessed set elements using iteration	☐	☐
5	Used membership operators correctly	☐	☐
6	Added an item using add()	☐	☐
7	Added multiple items using update() when required	☐	☐
8	Removed items using appropriate set methods	☐	☐
9	Used remove() and discard() appropriately	☐	☐
10	Applied set methods and keywords correctly	☐	☐
11	Applied union correctly	☐	☐
12	Applied intersection correctly	☐	☐
13	Applied difference correctly	☐	☐
14	Applied symmetric difference correctly	☐	☐
15	Used a set constructor appropriately	☐	☐
16	Applied built-in functions with sets	☐	☐
17	Determined set length correctly	☐	☐
18	Developed a program for storing unique data	☐	☐
19	Tested the Python program	☐	☐
20	Debugged syntax or logic errors when necessary	☐	☐
21	Verified the expected output	☐	☐

Comment:

Overall Assessment

Competent: ☐ Yes ☐ No

Needs Further Practice: ☐ Yes ☐ No

Trainer's Name: _____

Trainer's Signature: _____

Date: _____